

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №3
з дисципліни «Об'єктно-орієнтоване програмування»
на тему:
«Розробка інтерфейсу користувача»

Виконав:

студентка групи ІМ-55

Горбань А.О

Номер в списку: 6

Перевірив:

Рекечинський Д.О

Київ - 2026

Завдання:

1. Створити у середовищі розробки проект з ім'ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Розрахунок варіанта:

Номер варіанта для Лабораторної роботи №3 (Ж) обчислюється на основі базового номера з попередніх лабораторних робіт ($N_{\text{базовий}} = 6$) за формулою:

$$Ж = N_{\text{базовий}} + 1 = 6 + 1 = 7$$

Максимальна кількість об'єктів у масиві фігур розраховується як:

$$N = Ж + 100 = 7 + 100 = 107$$

Параметри графічного редактора за Варіантом Ж = 7:

Параметр	Значення та налаштування
Базовий номер ($N_{\text{базовий}}$)	6
Розрахунковий варіант (Ж)	7
Розмір масиву фігур (N)	107 об'єктів (MAX_SHAPES = 107)
Стиль «гумового» сліду	Пунктирна лінія (PS_DOT)
Заповнення прямокутника	Світло-зелений колір (RGB(144, 238, 144))
Заповнення еліпса	Порожнє / прозоре заповнення (NULL_BRUSH)

Елементи інтерфейсу	Головне меню, Toolbar із 4 кнопками, Tooltips
---------------------	---

Файл resource.h

```
#ifndef RESOURCE_H
#define RESOURCE_H

#define IDR_MAIN_MENU      101

#define IDB_TOOLBAR        102


#define IDM_POINT          201

#define IDM_LINE           202

#define IDM_RECTANGLE      203

#define IDM_ELLIPSE        204

#define IDM_EXIT           205

#endif
```

Файл resource.rc

```
#include <windows.h>
```

```
#include "resource.h"
```

```
#pragma code_page(65001)
```

```
IDR_MAIN_MENU MENU
```

```
BEGIN
```

```
    POPUP "Файл"
```

BEGIN

MENUITEM "Вихід", IDM_EXIT

END

POPUP "Об'єкти"

BEGIN

MENUITEM "Крапка", IDM_POINT

MENUITEM "Лінія", IDM_LINE

MENUITEM "Прямокутник", IDM_RECTANGLE

MENUITEM "Еліпс", IDM_ELLIPSE

END

END

IDB_TOOLBAR BITMAP "toolbar.bmp"

Файл shape.h

```
#ifndef SHAPE_H

#define SHAPE_H

#include <windows.h>

#include <cmath>

class Shape {

protected:

    long x1, y1, x2, y2;

public:

    virtual ~Shape() {}
```

```

    void SetPoints(long startX, long startY, long endX, long endY) {

        x1 = startX; y1 = startY;

        x2 = endX;    y2 = endY;

    }

    virtual void Draw(HDC hdc) = 0;
};

```

```

class PointShape : public Shape {

public:

    void Draw(HDC hdc) override {

        SetPixel(hdc, x1, y1, RGB(0, 0, 0));

    }

};

```

```

class LineShape : public Shape {

public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        MoveToEx(hdc, x1, y1, NULL);

        LineTo(hdc, x2, y2);
    }
};

```

```

        SelectObject(hdc, hOldPen);

        DeleteObject(hPen);

    }

};

class RectangleShape : public Shape {
public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hBrush = CreateSolidBrush(RGB(144, 238, 144)); //
Світло-зелене заповнення

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

        long dx = std::abs(x2 - x1);

        long dy = std::abs(y2 - y1);

        Rectangle(hdc, x1 - dx, y1 - dy, x1 + dx, y1 + dy);

        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

```

```

        DeleteObject(hBrush);

    }

};

class EllipseShape : public Shape {

public:

    void Draw(HDC hdc) override {

        HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

        HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        Ellipse(hdc, x1, y1, x2, y2);

        SelectObject(hdc, hOldPen);

        SelectObject(hdc, hOldBrush);

        DeleteObject(hPen);

    }

};

#endif

```

```
#ifndef _WIN32_IE
#define _WIN32_IE 0x0500
#endif

#ifndef EDITOR_H
#define EDITOR_H

#include <windows.h>
#include <commctrl.h>
#include "Shape.h"
#include "resource.h"

const int MAX_SHAPES = 107; // Варіант 7: N = 7 + 100 = 107

class Editor {
private:
    Shape* pcshape[MAX_SHAPES];

    int shapeCount;

    int currentTool;

    bool isDrawing;

    POINT startPoint;

    POINT currentPoint;

    HWND hWndParent; // Єдине найменування змінної
```

```

    HWND hToolbar;

    HPEN hRubberPen;

    void UpdateWindowTitle();

    void DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2);

public:
    Editor();

    ~Editor();

    void Init(HWND hWnd);

    void OnCommand(WPARAM wParam);

    LRESULT OnNotify(LPARAM lParam);

    void OnLButtonDown(HWND hWnd, LPARAM lParam);

    void OnMouseMove(HWND hWnd, LPARAM lParam);

    void OnLButtonUp(HWND hWnd, LPARAM lParam);

    void OnPaint(HWND hWnd);

};

#endif

```

Файл editor.cpp

```

#include "Editor.h"

Editor::Editor() {

```

```
    shapeCount = 0;

    currentTool = IDM_POINT;

    isDrawing = false;

    hToolbar = NULL;

    hWndParent = NULL;

    hRubberPen = NULL;

    for (int i = 0; i < MAX_SHAPES; ++i) {

        pcshape[i] = NULL;

    }

}

Editor::~Editor() {

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            delete pcshape[i];

            pcshape[i] = NULL;

        }

    }

    if (hRubberPen) {

        DeleteObject(hRubberPen);

    }

}

void Editor::Init(HWND hWnd) {
```

```
hWndParent = hWnd;
```

```
TBADDBITMAP tbab;
```

```
tbab.hInst = GetModuleHandle(NULL);
```

```
tbab.nID = IDB_TOOLBAR;
```

```
hToolbar = CreateToolBarEx(
```

```
hWnd,
```

```
WS_CHILD | WS_VISIBLE | TBSTYLE_FLAT | TBSTYLE_TOOLTIPS,
```

```
1, 5, GetModuleHandle(NULL), IDB_TOOLBAR,
```

```
NULL, 0, 16, 16, 16, 16, sizeof(TBBUTTON)
```

```
);
```

```
TBBUTTON tbb[4] = {
```

```
{ 0, IDM_POINT, TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0
```

```
},
```

```
{ 1, IDM_LINE, TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0
```

```
},
```

```
{ 2, IDM_RECTANGLE, TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0
```

```
},
```

```
{ 3, IDM_ELLIPSE, TBSTATE_ENABLED, TBSTYLE_BUTTON, {0}, 0, 0
```

```
}
```

```
};
```

```
SendMessage(hToolbar, TB_ADDBUTTONS, 4, (LPARAM)&tbb);
```

```
UpdateWindowTitle();
```

```
}
```

```
void Editor::UpdateWindowTitle() {
```

```
    const wchar_t* toolName = L"Крапка";
```

```
    switch (currentTool) {
```

```
        case IDM_POINT:    toolName = L"Крапка"; break;
```

```
        case IDM_LINE:     toolName = L"Лінія"; break;
```

```
        case IDM_RECTANGLE: toolName = L"Прямокутник"; break;
```

```
        case IDM_ELLIPSE:  toolName = L"Еліпс"; break;
```

```
    }
```

```
    wchar_t title[128];
```

```
    wprintf(title, L"Лабораторна 3 - [Поточний інструмент: %s]",  
toolName);
```

```
    SetWindowText(hWndParent, title);
```

```
}
```

```
void Editor::OnCommand(WPARAM wParam) {
```

```
    int id = LOWORD(wParam);
```

```
    if (id >= IDM_POINT && id <= IDM_ELLIPSE) {
```

```
        currentTool = id;
```

```
        UpdateWindowTitle();
```

```
    } else if (id == IDM_EXIT) {
```

```
        PostQuitMessage(0);
```

```
    }
```

```
}
```

```
LRESULT Editor::OnNotify(LPARAM lParam) {

    LPNMHDR pnmh = (LPNMHDR)lParam;

    if (pnmh->code == TTN_GETDISPINFO) {

        NMTTDISPINFOW* pttdi = (NMTTDISPINFOW*)lParam;

        switch (pttdi->hdr.idFrom) {

            case IDM_POINT:    pttdi->lpszText = (LPWSTR)L"Малювати
крапку"; break;

            case IDM_LINE:     pttdi->lpszText = (LPWSTR)L"Малювати
лінію"; break;

            case IDM_RECTANGLE: pttdi->lpszText = (LPWSTR)L"Малювати
прямокутник"; break;

            case IDM_ELLIPSE:  pttdi->lpszText = (LPWSTR)L"Малювати
еліпс"; break;

        }

    }

    return 0;
}

// Виділення пера 1 раз при DOWN (виправлення зауваження)

void Editor::OnLButtonDown(HWND hWnd, LPARAM lParam) {

    isDrawing = true;

    startPoint.x = LOWORD(lParam);

    startPoint.y = HIWORD(lParam);

    currentPoint = startPoint;
}
```

```
// Пунктирне чорне перо (7 mod 4 = 3)

hRubberPen = CreatePen(PS_DOT, 1, RGB(0, 0, 0));

SetCapture(hWnd);

}

void Editor::DrawRubberBand(HWND hWnd, POINT pt1, POINT pt2) {

    HDC hdc = GetDC(hWnd);

    int oldROP = SetROP2(hdc, R2_NOTXORPEN);

    HPEN hOldPen = (HPEN)SelectObject(hdc, hRubberPen);

    if (currentTool == IDM_LINE) {

        MoveToEx(hdc, pt1.x, pt1.y, NULL);

        LineTo(hdc, pt2.x, pt2.y);

    } else if (currentTool == IDM_RECTANGLE) {

        long dx = std::abs(pt2.x - pt1.x);

        long dy = std::abs(pt2.y - pt1.y);

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        Rectangle(hdc, pt1.x - dx, pt1.y - dy, pt1.x + dx, pt1.y + dy);

        SelectObject(hdc, hOldBrush);

    } else if (currentTool == IDM_ELLIPSE) {

        HBRUSH hNullBrush = (HBRUSH)GetStockObject(NULL_BRUSH);

        HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hNullBrush);

        Ellipse(hdc, pt1.x, pt1.y, pt2.x, pt2.y);

    }

}
```

```
        SelectObject(hdc, hOldBrush);

    }

    SelectObject(hdc, hOldPen);

    SetROP2(hdc, oldROP);

    ReleaseDC(hWnd, hdc);
}

void Editor::OnMouseMove(HWND hWnd, LPARAM lParam) {

    if (!isDrawing || currentTool == IDM_POINT) return;

    DrawRubberBand(hWnd, startPoint, currentPoint);

    currentPoint.x = LOWORD(lParam);

    currentPoint.y = HIWORD(lParam);

    DrawRubberBand(hWnd, startPoint, currentPoint);
}

// Видалення пера в UP

void Editor::OnLButtonUp(HWND hWnd, LPARAM lParam) {

    if (!isDrawing) return;

    if (currentTool != IDM_POINT) {

        DrawRubberBand(hWnd, startPoint, currentPoint);

    }
}
```

```
ReleaseCapture();

isDrawing = false;

if (hRubberPen) {

    DeleteObject(hRubberPen);

    hRubberPen = NULL;

}

if (shapeCount >= MAX_SHAPES) return;

POINT endPoint = { LOWORD(lParam), HIWORD(lParam) };

Shape* newShape = NULL;

switch (currentTool) {

    case IDM_POINT:      newShape = new PointShape(); break;

    case IDM_LINE:       newShape = new LineShape(); break;

    case IDM_RECTANGLE:  newShape = new RectangleShape(); break;

    case IDM_ELLIPSE:    newShape = new EllipseShape(); break;

}

if (newShape) {

    newShape->SetPoints(startPoint.x, startPoint.y, endPoint.x,
endPoint.y);

    pcshape[shapeCount++] = newShape;

    InvalidateRect(hWnd, NULL, TRUE);

}
```

```

}

void Editor::OnPaint(HWND hWnd) {

    PAINTSTRUCT ps;

    HDC hdc = BeginPaint(hWnd, &ps);

    for (int i = 0; i < shapeCount; ++i) {

        if (pcshape[i]) {

            pcshape[i]->Draw(hdc);

        }

    }

    EndPaint(hWnd, &ps);

}

```

Файл `main.cpp`

```

#include <windows.h>

#include <commctrl.h>

#include "Editor.h"

#include "resource.h"

Editor g_editor;

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {

    switch (message) {

        case WM_CREATE:

```

```
InitCommonControls();
```

```
g_editor.Init(hWnd);
```

```
break;
```

```
case WM_COMMAND:
```

```
g_editor.OnCommand(wParam);
```

```
break;
```

```
case WM_NOTIFY:
```

```
return g_editor.OnNotify(lParam);
```

```
case WM_LBUTTONDOWN:
```

```
g_editor.OnLButtonDown(hWnd, lParam);
```

```
break;
```

```
case WM_MOUSEMOVE:
```

```
g_editor.OnMouseMove(hWnd, lParam);
```

```
break;
```

```
case WM_LBUTTONUP:
```

```
g_editor.OnLButtonUp(hWnd, lParam);
```

```
break;
```

```
case WM_PAINT:
```

```
g_editor.OnPaint(hWnd);
```

```

        break;

        case WM_DESTROY:

            PostQuitMessage(0);

            break;

        default:

            return DefWindowProc(hWnd, message, wParam, lParam);

    }

    return 0;
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR
lpCmdLine, int nCmdShow) {

    WNDCLASSEX wc = { sizeof(WNDCLASSEX) };

    wc.style          = CS_HREDRAW | CS_VREDRAW;

    wc.lpfnWndProc    = WndProc;

    wc.hInstance      = hInstance;

    wc.hCursor        = LoadCursor(NULL, IDC_ARROW);

    wc.hbrBackground  = (HBRUSH)(COLOR_WINDOW + 1);

    wc.lpszMenuName    = MAKEINTRESOURCE(IDR_MAIN_MENU);

    wc.lpszClassName  = L"Lab3WindowClass";

    RegisterClassEx(&wc);

    HWND hWnd = CreateWindowEx(

```

```

    0, L"Lab3WindowClass", L"Лабораторна 3",
    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT,
    800, 600, NULL, NULL, hInstance, NULL
);

ShowWindow(hWnd, nCmdShow);

UpdateWindow(hWnd);

MSG msg;

while (GetMessage(&msg, NULL, 0, 0)) {

    TranslateMessage(&msg);

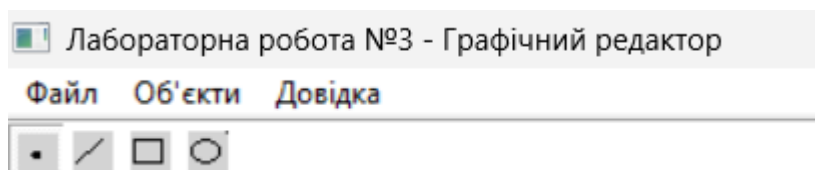
    DispatchMessage(&msg);

}

return (int)msg.wParam;
}

```

Скріншоти:



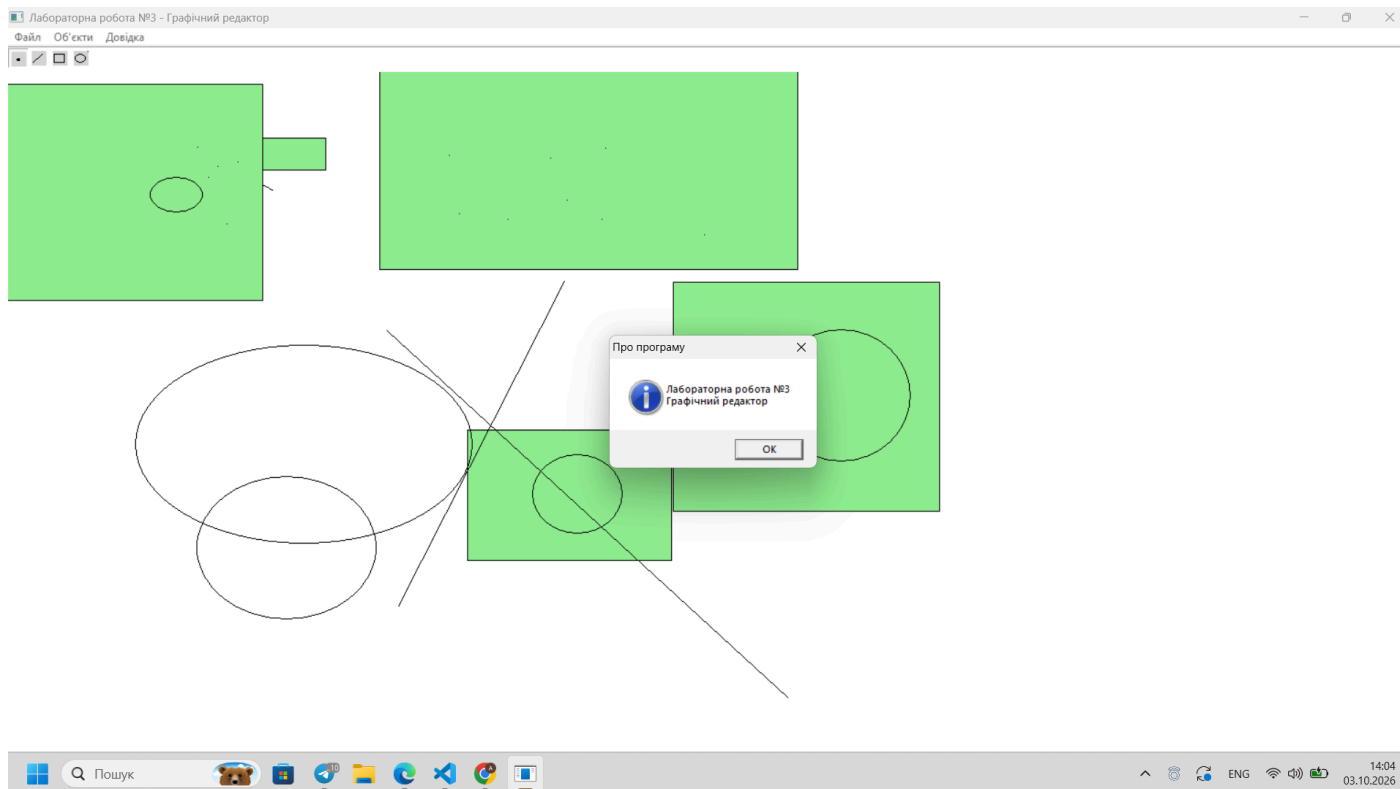
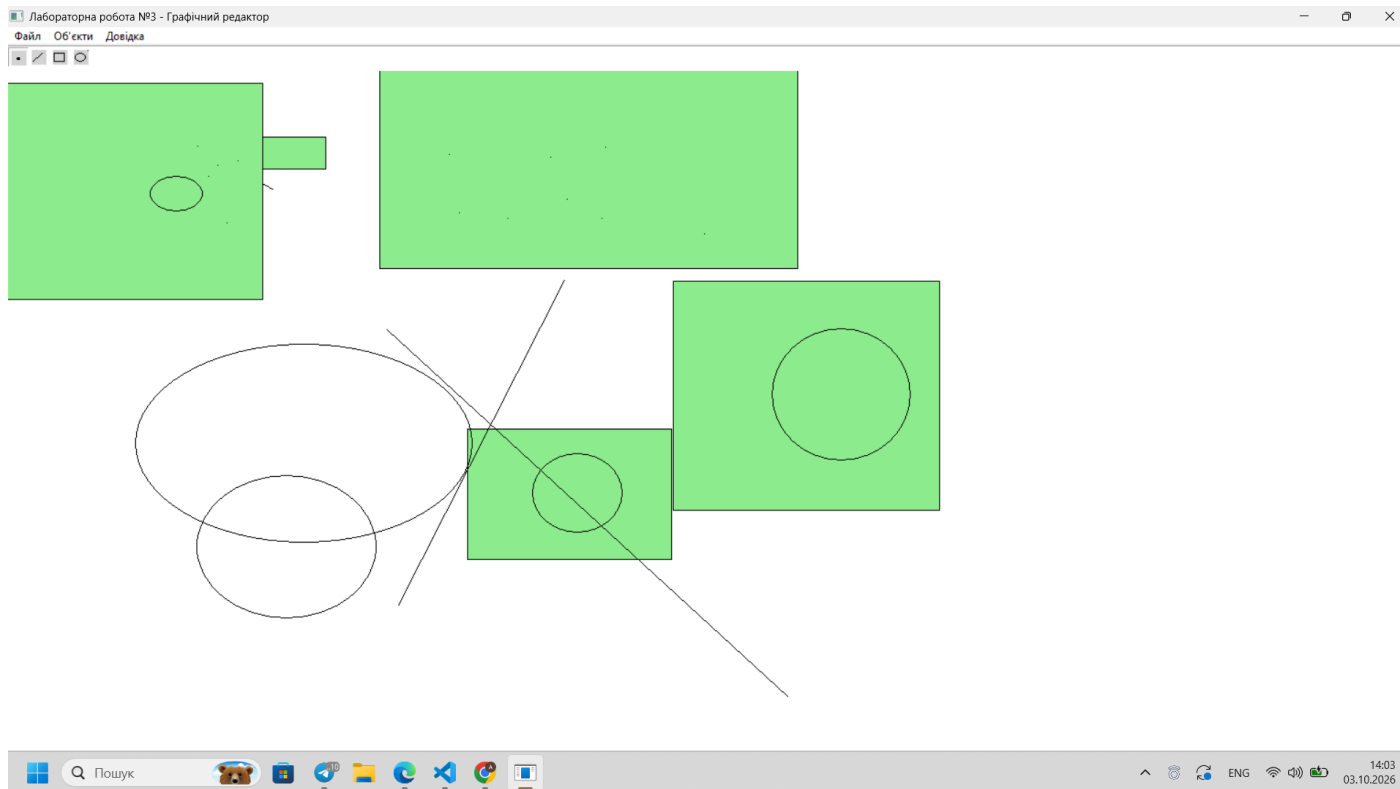
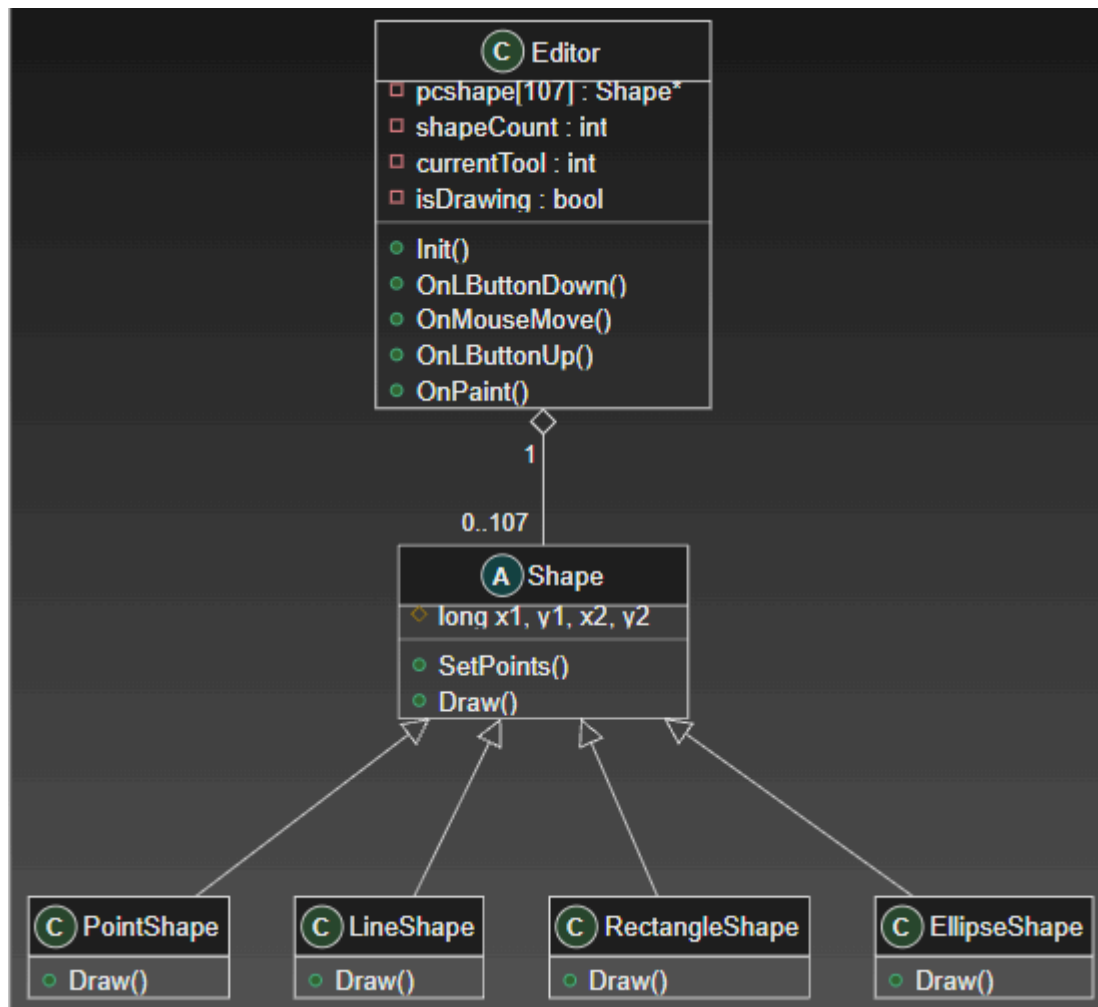


Схема успадкування класів – діаграма класів:



Висновки:

У ході виконання лабораторної роботи №3 було створено графічний редактор Win32 API із застосуванням принципів об'єктно-орієнтованого програмування. Реалізовано ієрархію класів фігур на чолі з абстрактним базовим класом Shape, розроблено клас Editor для керування станом редактора та масивом об'єктів. Додаток підтримує інтерактивну панель інструментів (Toolbar) з підказками (Tooltips) та відображення «гумового» сліду. Усі вимоги варіанта 7 ($N = 107$, пунктирний слід, заповнений прямокутник та порожній еліпс) виконуються в повному обсязі, витoki ресурсів GDI відсутні